

Chapter 16

Disk Storage, Basic File Structures, and Hashing, and Modern Storage Architectures

1

Chapter Outline

- Disk Drives
- Files of Records
- Unordered Files
- Ordered Files
- Hashed Files
 - Static Hashed File
 - Dynamic Hashing Techniques

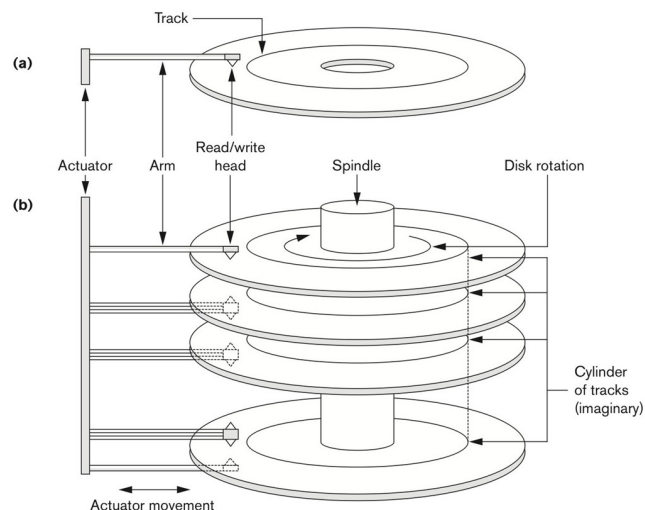
2

Disk Drives

- Preferred secondary storage device for high storage capacity and low cost.
- Data stored as magnetized areas on magnetic disk surfaces.
- A disk pack (i.e., disk drive) contains several magnetic disks connected to a rotating spindle.
- Disks are divided into concentric circular tracks on each disk surface.
 - Track capacities vary typically from tens of Kbytes to hundreds of Kbytes (or more).

3

Disk Drives (cont'd)



4

Disk Storage Devices (cont'd)

- Tracks are divided into smaller sectors.
- The division of a track into sectors is hard-coded on the disk surface and cannot be changed.
 - Typical sector size are 512 bytes, 1K bytes, 2K bytes, and 4K bytes.
- A track can be divided into equal-sized blocks during the formatting.
 - The block size is fixed for each system.
 - Typical block sizes range from 512 bytes to 8K bytes.
 - Whole blocks are transferred between disk and main memory for processing.

5

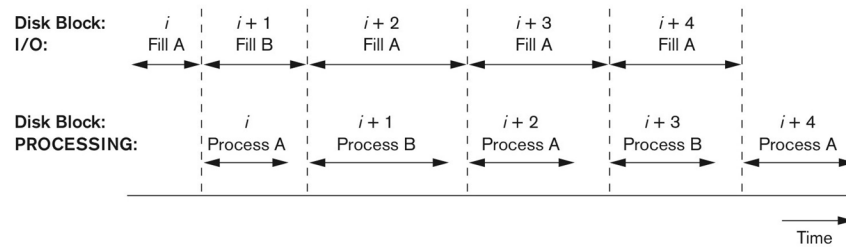
Disk Storage Devices (cont'd)

- A read-write head moves to the track that contains the block to be transferred ([seek time](#)).
 - Disk rotation moves the block under the read-write head for reading or writing ([rotational latency](#)).
- A disk block address consists of:
 - a [cylinder number](#): a cylinder is an imaginary collection of tracks of same radius from all recorded surfaces.
 - the [track number \(or surface number\)](#) within the cylinder.
 - and [block number](#) within the track.
- Reading or writing a disk block is time consuming because of the [seek time](#) and [rotational latency \(aka latency\)](#).
- [Double buffering](#) can be used (in main memory) to speed up the transfer of disk blocks into main memory.

6

Double Buffering of Blocks

Figure 16.4 Use of two buffers, A and B, for reading from disk.



7

Typical Disk Parameters

Table 16.2 Specifications of Typical High-End Enterprise Disks from Seagate (a) Seagate Enterprise Performance 10 K HDD - 1200 GB

Specifications	1200GB
SED Model Number	ST1200MM0017
SED FIPS 140-2 Model Number	ST1200MM0027
Model Name	Enterprise Performance 10K HDD v7
Interface	6Gb/s SAS
Capacity	
Formatted 512 Bytes/Sector (GB)	1200
External Transfer Rate (MB/s)	600
Performance	
Spindle Speed (RPM)	10K
Average Latency (ms)	2.9
Sustained Transfer Rate Outer to Inner Diameter (MB/s)	204 to 125
Cache, Multisegmented (MB)	64

8

Typical Disk Parameters (cont'd)

Table 16.2 Specifications of Typical High-End Enterprise Disks from Seagate (a) Seagate Enterprise Performance 10 K HDD - 1200 GB

Configuration/Reliability

Disks	4
Heads	8
Nonrecoverable Read Errors per Bits Read	1 per 10E16
Annualized Failure Rate (AFR)	0.44%

Physical

Height (in/mm, max)	0.591/15.00
Width (in/mm, max)	2.760/70.10
Depth (in/mm, max)	3.955/100.45
Weight (lb/kg)	0.450/0.204

Courtesy Seagate Technology

9

Typical Disk Parameters (cont'd)

Table 16.2 (b) Internal Drive Characteristics of 300 GB–900 GB Seagate Drives

	ST900MM0006	ST600MM0006	ST450MM0006	ST300MM0006	
	ST900MM0026	ST600MM0026	ST450MM0026	ST300MM0026	
	ST900MM0046	ST600MM0046	ST450MM0046	ST300MM0046	
	ST900MM0036				
Drive capacity	900	600	450	300	GB (formatted, rounded off value)
Read/write data heads	6	4	3	2	
Bytes per track	997.9	997.9	997.9	997.9	KBytes (avg, rounded off values)
Bytes per surface	151,674	151,674	151,674	151,674	MB (unformatted, rounded off value)
Tracks per surface (total)	152	152	152	152	KTracks (user accessible)
Tracks per inch	279	279	279	279	KTPI (average)
Peak bits per inch	1925	1925	1925	1925	KBPI
Areal density	538	538	538	538	Gb/in ²
Disk rotation speed	10K	10K	10K	10K	rpm
Avg rotational latency	2.9	2.9	2.9	2.9	ms

Records

- Each record consists of a related data values or items, where each value corresponds to a particular field of the record.
 - A tuple in a relation is stored as a record, as the relation is stored as a file. Each field in the record corresponds to an attribute.
- Fixed-length and variable-length records
 - In a file of fixed-length records, all records have the same format.
 - Files of variable-length records require additional information to be stored in each record, such as separator characters and field types.
- Fields themselves may be fixed-length or variable-length
 - In case of having variable-length fields, separator characters or field lengths are needed so that the record can be “parsed.”
- Records can be unspanned or spanned
 - Unspanned: no record can span two blocks.
 - Spanned: a record can be stored in more than one block.

11

Fixed- and Variable-length Records

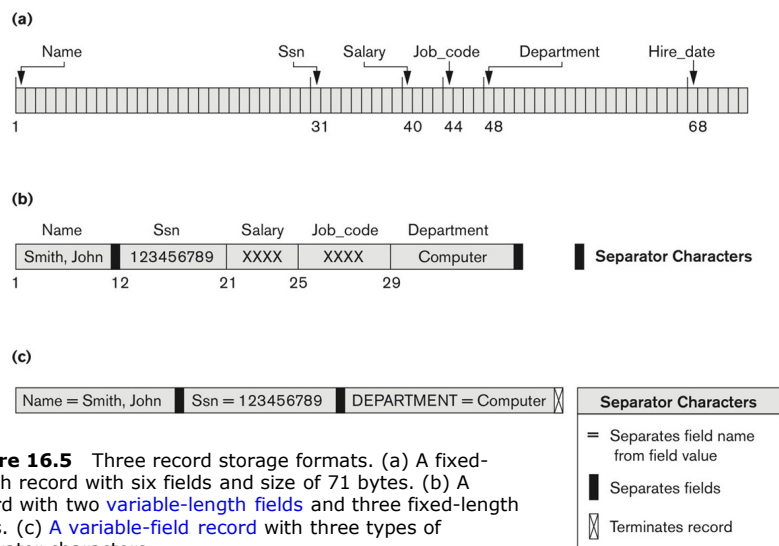
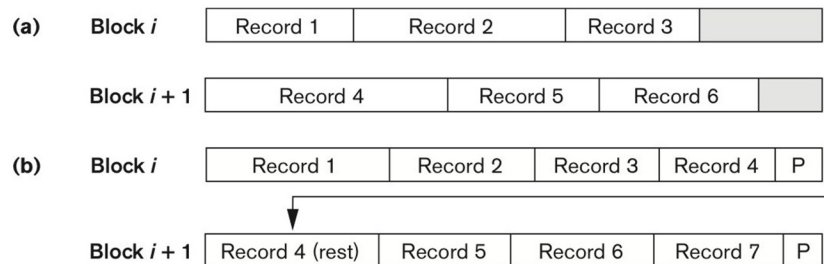


Figure 16.5 Three record storage formats. (a) A fixed-length record with six fields and size of 71 bytes. (b) A record with two variable-length fields and three fixed-length fields. (c) A variable-field record with three types of separator characters.

12

Unspanned and Spanned Records

Figure 16.6 Types of record organization. (a) Unspanned. (b) Spanned.



13

Files of Records

- A file is a *sequence of records*, where each record is a collection of data values (or data items).
- A *file descriptor* (or *file header*) includes information to determine the disk addresses of the file blocks as well as record format descriptions, which may include:
 - field lengths and the order of fields for fixed-length records, and
 - field type codes and separator characters for variable-length records.
- Records are stored in disk blocks.
- The *blocking factor* for a file is the maximum number of file records that can be stored in a disk block.

14

Operations on Files

- Typical file operations include:
 - **Open**: Prepares the file for access, allocates buffers to hold file blocks, retrieves the file header, and sets the file pointer (aka *current record pointer*) to the beginning of the file.
 - **Find (or Locate)**: Searches for the first file record that satisfies a search condition, and makes it the current record.
 - **FindNext**: Searches for the next file record (from the current record) that satisfies the search condition, and makes it the current file record.
 - **Read (or Get)**: Reads the current file record from the buffer into a program variable.
 - **Insert**: Inserts a new record into the file and makes it the current file record.
 - **Delete**: Removes the current file record from the file, usually by setting the *deletion marker* of the record to indicate that it is no longer valid.
 - **Modify**: Changes the values of some fields of the current file record.
 - **Close**: Terminates access to the file by releasing the buffers and perform any other needed cleanup operations.
 - **Reorganize**: Reorganizes the file records.
 - For example, the records marked deleted are physically removed from the file (i.e., *records are packed*) or records are sorted (if necessary).
 - **FindOrdered**: retrieves all the records of the file in the order of the values of a specific field of the file.

15

Unordered Files

- Records are not ordered based on any field in the file.
- New records are inserted at the end of the file (i.e., appended) thus a *record insertion is quite efficient*.
 - The last disk block of the file is read into a buffer, the new record is added, and the block is written back to disk.
- An extra bit, called a *deletion marker*, can be stored with each record. In that case, we delete a record by setting the deletion marker to a certain value.
- During the reorganization, the file blocks can be accessed sequentially and records are packed whenever unused space exists in a block.
- A *linear search* through the file records is necessary to search for a record.
- Reading the records in the order of the values of a particular field requires the sorting of records.

16

Ordered Files

- File records are kept sorted by their values of an *ordering field*.
- Insertion is expensive: a new record must be inserted at a specific position in the file.
 - It is common to keep a separate unordered *overflow file* (or transaction file). New records are appended to the overflow file, which is periodically (sorted then) merged with the main ordered file (called the *master file*).
- A *binary search* can be used to search for a record on its ordering field value.
 - This requires reading and searching $\log_2$ of the number of file blocks on the average.
- Reading the records in the order of the ordering field is quite efficient because no sorting is required.

17

Ordered File Example

Figure 16.7 Some blocks of an ordered (sequential) file of EMPLOYEE records with Name as the ordering key field.

	Name	Sex	Birth_date	Job	Salary	Sex
Block 1	Aaron, Ed					
	Abbott, Diane					
	⋮					
Block 2	Acosta, Marc					
	Adams, John					
	Adams, Robin					
Block 3	⋮					
	Akers, Jan					
	⋮					
Block 4	Alexander, Ed					
	Alfred, Bob					
	⋮					
Block 5	Allen, Sam					
	Allen, Troy					
	Anders, Keith					
Block 6	⋮					
	Anderson, Rob					
	⋮					
Block 7	Anderson, Zach					
	Angeli, Joe					
	⋮					
Block 8	Archer, Sue					
	Arnold, Mack					
	Arnold, Steven					
Block 9	⋮					
	Atkins, Timothy					
	⋮					
Block n-1	Wong, James					
	Wood, Donald					
	⋮					
Block n	Woods, Manny					
	Wright, Pam					
	Wyatt, Charles					
Block n+1	⋮					
	Zimmer, Byron					
	⋮					

18

Static Hashed Files

- One field of a file is designated to be the **hash key** of the file.
- The record with hash key value K is stored in bucket i , where $i=h(K)$ is the bucket number, and h is the **hashing function** used for the file.
 - A **bucket is a logical block** that can be stored in one or more disk blocks, but the bucket size is fixed.
 - Hashing the records of a file into disk blocks is called **external hashing**.
- Searching is very efficient on the hash key.
- **Collision** occurs when a new record hashes to a bucket that is already full.
 - An **overflow file** (aka **overflow area**) is kept for storing such records.
 - **Overflow chaining**: Maintaining a pointer in each bucket to a linked list of overflow records (stored in the overflow area) for the bucket.

19

Static Hashed File — Overflow Chaining

K : hash key value
of a record.

Hash value of the record:

$$h(K) = K \bmod 10$$

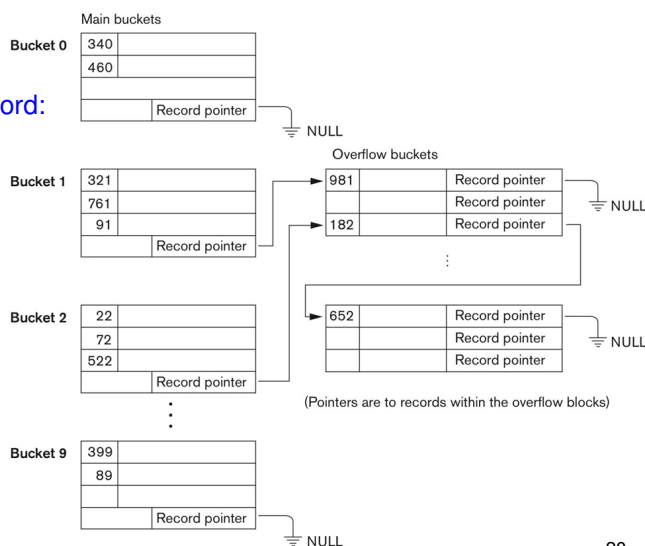


Figure 16.10
Handling overflow for
buckets by chaining.

20

Static Hashed Files (cont'd)

- Main disadvantages of static external hashing:
 - A fixed number of blocks are allocated to a file. So it is a problem if the number of records in the file grows or shrinks a lot.
 - Underestimation of the file size: If the actual number of records is larger than the maximum number of records that can be stored in the fixed number of blocks allocated, numerous collisions will occur and the retrieval will be slowed down because of the long lists of overflow records.
 - Overestimation of the file size: If the actual number of records is substantially less than the maximum number of records that can be stored in the fixed number of blocks allocated, the storage space utilization becomes low.
- Main disadvantages of hashed files (both static and dynamic):
 - Range queries and ordered retrieval requires the sorting of records because the records of a hashed file are not ordered on any field.

21

Static Hashed Files (cont'd)

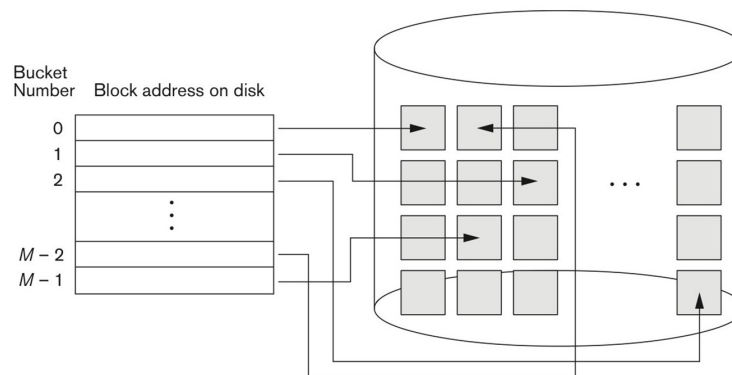


Figure 16.9 Matching bucket numbers to disk block addresses.

22

Dynamic Hashed Files

- Hashing techniques are adapted to allow the dynamic growth and shrinking of the number of blocks.
 - These techniques include **dynamic hashing**, **extendible hashing**, and **linear hashing**.
- Both dynamic and extendible hashing use the **binary representation** (i.e., **bit string**) of the hash value $h(K)$ in order to access the **directory**.
 - In dynamic hashing, the directory is a binary tree.
 - In extendible hashing, the directory is an array of entries (i.e., a table of entries).

23

Dynamic Hashed Files (cont'd)

- Dynamic hashing and extendible hashing do not require an overflow area. If a record is hashed to a bucket that is full, it causes the bucket to split into two buckets and the records in that overflowed bucket are redistributed among the two buckets.
 - The directory should be updated appropriately when an overflowed bucket is split into two buckets.
- Linear hashing does require an overflow area but does not use a directory.
 - Buckets are split in *linear order* — one at a time — as bucket overflows occur.

24

Dynamic Hashing

- A binary tree structured directory has two types of nodes: **internal node** and **leaf node**.
 - Internal node has a left pointer corresponding to the “0” bit and a right pointer corresponding to the “1” bit at the current bit position used in the hash value.
 - Each leaf node holds a pointer to a bucket.
- **Each record is hashed to a bucket based on the leftmost (or rightmost) bits of the hash value.**
- If a bucket overflows, it is split into two buckets, and the records are distributed based on the next unused leftmost (or rightmost) bit in their hash values.

25

Dynamic Hashing Example

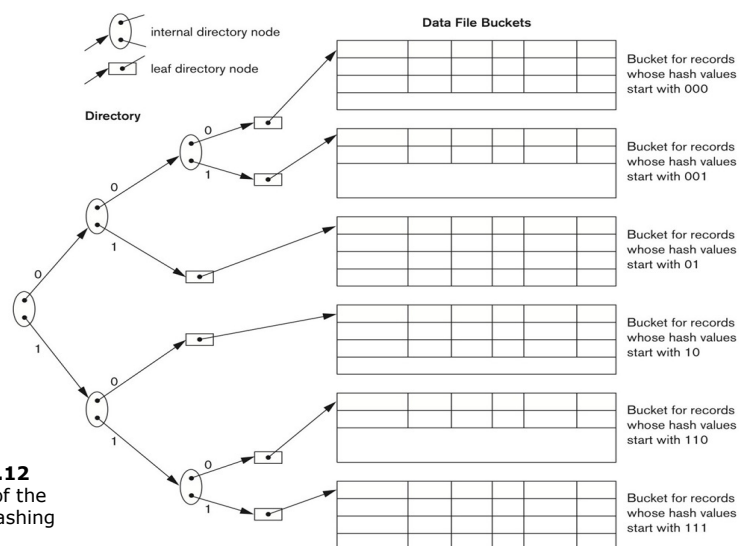


Figure 16.12
Structure of the
dynamic hashing
scheme.

26

Extendible Hashing

- Maintains a table-form directory — an array of 2^d entries. d is called the *global depth* of the directory, and it represents the bit length of the address (i.e., size of the address) of the directory entries.
- The first d bits of a hash value determine the address of the directory entry to look up, and the directory entry stores the corresponding bucket address.
- Several adjacent (i.e., consecutive) directory entries with the same first d' bits (where $d' < d$) in their addresses may contain the same bucket address if all the records that hash to these entries fit in a single bucket.

27

Extendible Hashing (cont'd)

- The *local depth* d' of each bucket represents that the records hashed to that bucket has common first d' bits in their hash values.
- If the local depth d' of a bucket is same as the global depth d (i.e., $d' = d$) the bucket is pointed to by only one entry in the directory.
 - The address of the entry is the same as the common d' bits of the hash values of the records stored in the bucket.
- For a bucket with the local depth $d' < d$, there are $2^{d-d'}$ consecutive directory entries pointing to that bucket.
 - These directory entries have the same first d' bits in their addresses.
- If a bucket that overflows has a local depth equal to the global depth of the directory (i.e., $d' = d$), then the size of the directory must be doubled, because we must use an extra bit (in the hash value) to distinguish the two buckets split from the overflowed bucket.

28

Extendible Hashing Example

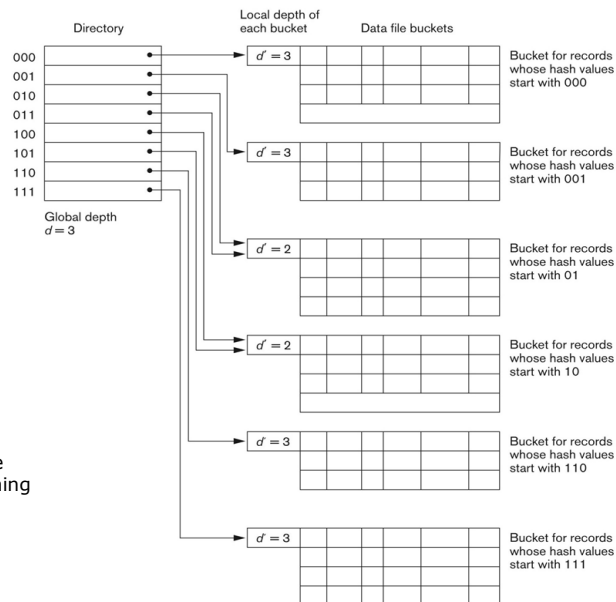


Figure 16.11
Structure of the
extendible hashing
scheme.

29

Extendible Hashing (cont'd)

- Two sibling buckets split from the same bucket are merged back into the original bucket if the records in those two buckets can be stored in a single bucket as a record has been removed from either one of the two sibling buckets.
- If the local depth d' of every bucket is smaller than the current global depth d of the directory, the directory can be reduced to half (i.e., back to the previous directory before doubling), and d is decremented by 1.

30

Linear Hashing

- The file starts with $M (= 2^m)$ buckets numbered 0, 1, 2, ..., $M-1$ and uses the $\text{mod } 2^m$ for hashing:
$$h_0(K) = K \bmod 2^m$$
 - $K \bmod 2^m$ is the value of the rightmost m bits of $(K)_2$.
- If a bucket overflows, the first bucket, Bucket 0, is split into two buckets — the original Bucket 0 and a new Bucket M — by using
$$h_1(K) = K \bmod 2^{m+1}$$
- **Overflow is handled by overflow chaining:**
overflowed records are linked to the bucket from which they were overflowed, unless that bucket happens to be the one to be split (at the time of the overflow).

31

Linear Hashing (cont'd)

- As overflow records occur, additional buckets are split in the linear order: 1, 2, 3, ..., $M-1$.
- **The value of a variable n is used to denote the next bucket to split.** So, the initial value of n is 0, and it is increased by 1 at every overflow, up to $M-1$.
- **For retrieving a record with a hash key value K , we first apply the hash function h_0 to K . If $h_0(K) < n$, then we use the next hash function h_1 to K because the original bucket (whose number is $h_0(K)$) has been already split into two buckets.**

32

Linear Hashing (cont'd)

- When $n = M$, original M buckets have been split. At this point, n is reset to 0, and each of the $2M$ buckets (Buckets 0, 1, 2, ..., $2M-1$) would be split, one by one, at each overflow using the next hash function $h_2(K) = K \bmod 2^{m+2}$.
- During the expansion of M buckets to $2M$ buckets, we need to maintain two hash functions h_0 and h_1 . And During the expansion of $2M$ buckets to $4M$ buckets, we need to maintain two hash functions h_1 and h_2 .

33

Linear Hashing (cont'd)

- Buckets can be merged based on the value of the **loading factor**, which represents the average number of records per bucket.
 - The last pair of buckets split (from a single bucket) are merged back to the original single bucket. In other words, the last bucket is merged back the original bucket from which it was split.
- Problems of Linear Hashing:
 - A bucket to split may not be an overflowed one.
 - A pair of buckets to be merged may not be underutilized ones.

34

Differences between Extendible Hashing (ExHash) and Linear Hashing (LH)

- LH does not use a directory, while ExHash uses a table-form directory.
- LH uses rightmost bits of hash values (as *mod* function is used for hashing), whereas ExHash can use either leftmost or rightmost bits of hash values.
- LH allows an overflow chain of records for each bucket, which may slow down the retrieval, because not only the records inside the bucket but also the overflowed records should be checked to find matching ones.
- Non-uniform distribution of the hash values (of records) will cause frequent overflows. In that case, LH may have low bucket utilization, whereas ExHash may have low directory utilization.

35